

Exercício

O objetivo do exercício é gerenciar um sistema de catálogo geral de faixas musicais de uma plataforma de streaming. As gravadoras fornecem ao sistema as seguintes informações: código da faixa (chave primária), nome da faixa, nome do artista, gênero musical. O objetivo é levantar dados sobre o catálogo. O arquivo a ser criado deve ser de registros e campos de tamanho variável, com um inteiro (4 bytes) no início do registro indicando o tamanho do registro, e com campos separados pelo caractere |.

Código da Faixa	Nome da Faixa	Artista	Gênero Musical
4 caracteres (fixo)	60 caracteres (máximo)	50 caracteres (máximo)	20 caracteres (máximo)

Ex.: 7042|Águas de Março|Elis Regina|MPB

As seguintes operações deverão estar disponíveis:

1. Inserção
2. Remoção
3. Compactação
4. Dump Arquivo
5. Carrega Arquivos (dependente da implementação)

Inserção (1)

Ao adicionar o registro de uma faixa, vocês terão que percorrer a lista de espaços disponíveis verificando se o novo registro se encaixa em algum dos espaços (vide Opção 2). Para tanto, usem a estratégia **best-fit** e, para facilitar, podem considerar fragmentação interna. Caso nenhum elemento da lista supra o espaço necessário para o novo registro, acrescente-o no final do arquivo. Os dados a serem inseridos devem ser recuperados de um arquivo a ser fornecido no momento da execução do programa (vide Opção 4).

Remoção (2)

Dado o “Código” da faixa (recuperado de um arquivo a ser fornecido no momento da execução do programa (vide Opção 4)), realize a remoção do respectivo registro. A remoção deve ser feita diretamente no arquivo de dados. Para reaproveitar o espaço removido vocês terão que acrescentar no arquivo uma lista ligada entre os espaços disponíveis. Assim, vocês terão que acrescentar as seguintes informações no arquivo:

1. Criem um registro cabeçalho e nele um campo que indica o offset para o primeiro elemento da lista.
2. Ao remover um registro, substitua-o no arquivo por: <tamanho em bytes do registro removido>@<offset para o próximo elemento da lista>, onde @ é um marcador indicando que este espaço está disponível (esse formato é apenas uma sugestão).
3. Um novo espaço disponível deve ser acrescentado sempre **no final** da lista (e não no início). Logo, vocês devem percorrer a lista até o último elemento e atualizar o offset dele para apontar para o novo espaço.

4. O final da lista é indicado por -1 no campo offset para o próximo elemento.

Compactação (3)

A estratégia de remoção vai criar fragmentos (internos e externos). Reconstruam o arquivo, quando solicitado pelo usuário, compactando todos os registros e limpando esses fragmentos (internos e externos).

Dump Arquivo (4)

Realiza o dump do arquivo.

Carrega Arquivos (5)

A fim de facilitar os testes, serão fornecidos dois arquivos: (a) "insere.bin" e (b) "remove.bin". O primeiro (a) conterà os dados a serem inseridos durante os testes (não necessariamente todos os dados serão inseridos). Para tanto, uma sugestão é carregar o arquivo em memória (um vetor de struct, por exemplo) e ir acessando cada posição conforme as inserções vão ocorrendo. Note que é possível encerrar a execução e recomeçar a execução, sendo necessário marcar, de algum modo, quantos registros já foram utilizados do mesmo.

Em relação a (b), o arquivo conterà uma lista de "Códigos" a serem utilizados durante a remoção. A ideia é a mesma já descrita, ou seja, carregar o arquivo em memória (um vetor de struct, por exemplo) e ir acessando cada posição conforme as remoções vão ocorrendo. Note que é possível encerrar a execução e recomeçar a execução, sendo necessário marcar, de algum modo, quantos registros já foram utilizados do mesmo.

Observações

1. Não criar o arquivo toda vez que o programa for aberto (fazer verificação).
2. O arquivo deve ser manipulado totalmente em memória secundária!